

Computational Economics

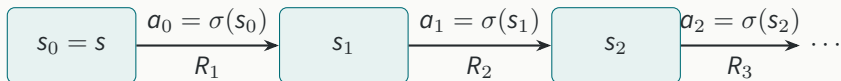
Lecture: Value Functions and the Bellman Equation

Discounted rewards, policy evaluation, and value iteration

Yaolang Zhong yaolang.zhong@e.u-tokyo.ac.jp
Graduate School of Economics, The University of Tokyo

From a State and a Policy to a Reward Stream

Start at $s_0 = s$ and follow a stationary deterministic policy $a_t = \sigma(s_t)$.



The **discounted accumulated reward**, or **return**, along this path is

$$G_0 = R_1 + \beta R_2 + \beta^2 R_3 + \dots = \sum_{t=0}^{\infty} \beta^t R_{t+1}, \quad 0 < \beta < 1.$$

- ▶ R_{t+1} is the reward from the action taken at t , as in the MDP timing.
- ▶ Fixing s and σ determines a probability law over paths. The return can still be random.

Discounting and a Finite Return

- ▶ One unit received now has weight 1; one period later it has weight β . A larger β places more weight on the future.
- ▶ Assume rewards are uniformly bounded: $|R_{t+1}| \leq \bar{R} < \infty$ almost surely, under every feasible policy. Then

$$|G_0| \leq \sum_{t=0}^{\infty} \beta^t \bar{R} = \frac{\bar{R}}{1 - \beta}.$$

- ▶ Keeping only the first H rewards leaves a bounded tail:

$$\left| G_0 - \sum_{t=0}^{H-1} \beta^t R_{t+1} \right| \leq \frac{\beta^H \bar{R}}{1 - \beta}.$$

For example, a reward of 1 every period has return $1/(1 - \beta) = 10$ when $\beta = 0.9$.

Example: Two Cake-Eating Policies

Example: Cake Eating

Start with two units of cake. For this example, restrict stock and consumption to whole units:

$$k \in \{0, 1, 2\}, \quad c \in \{0, \dots, k\}, \quad k' = k - c, \quad r(k, c) = \sqrt{c}, \quad \beta = 0.9.$$

Consume everything: $\sigma_A(k) = k$

$$k_t : \quad 2 \longrightarrow 0 \longrightarrow 0$$

$$R_{t+1} : \quad \sqrt{2}, 0, 0, \dots$$

$$G_0 = \sqrt{2} \approx 1.414.$$

One unit at a time: $\sigma_B(k) = \min\{1, k\}$

$$k_t : \quad 2 \longrightarrow 1 \longrightarrow 0$$

$$R_{t+1} : \quad 1, 1, 0, \dots$$

$$G_0 = 1 + 0.9 = 1.9.$$

A smaller reward today can produce a larger accumulated reward. These paths are deterministic, so their returns can be calculated directly.

The Value Function of a Policy

The **value function under policy** σ assigns each starting state its expected return:

$$V^\sigma(s) = \mathbb{E}_\sigma[G_0 \mid s_0 = s] = \mathbb{E}_\sigma \left[\sum_{t=0}^{\infty} \beta^t R_{t+1} \mid s_0 = s \right].$$

- ▶ s fixes the initial state; σ specifies actions at every future state. The expectation averages over the resulting paths and rewards.
- ▶ Since $r(s, a)$ is the conditional mean reward, boundedness also gives

$$V^\sigma(s) = \mathbb{E}_\sigma \left[\sum_{t=0}^{\infty} \beta^t r(s_t, \sigma(s_t)) \mid s_0 = s \right].$$

- ▶ For the cake example, $V^{\sigma_A}(2) = \sqrt{2}$ and $V^{\sigma_B}(2) = 1.9$.

The Return Has a Recursive Form

From any date t , define the return using that date as the discounting origin:

$$G_t = R_{t+1} + \beta R_{t+2} + \beta^2 R_{t+3} + \dots$$

Separate the first reward from all later rewards:

$$\begin{aligned} G_t &= R_{t+1} + \beta(R_{t+2} + \beta R_{t+3} + \dots) \\ &= R_{t+1} + \beta G_{t+1}. \end{aligned}$$

- ▶ The future return is the same object, starting one period later.
- ▶ With time-homogeneous dynamics and a stationary policy,

$$\mathbb{E}_\sigma[G_{t+1} \mid s_{t+1} = s'] = V^\sigma(s').$$

This lets us replace an infinite reward sequence with a one-step equation.

The Bellman Equation for a Fixed Policy

Take conditional expectations of $G_0 = R_1 + \beta G_1$:

$$\begin{aligned} V^\sigma(s) &= \mathbb{E}_\sigma[R_1 + \beta G_1 \mid s_0 = s] \\ &= r(s, \sigma(s)) + \beta \mathbb{E}[V^\sigma(s_1) \mid s_0 = s, a_0 = \sigma(s)]. \end{aligned}$$

The second equality uses the law of iterated expectations and the Markov structure.

For a finite state space, the **policy Bellman equation** is

$$V^\sigma(s) = r(s, \sigma(s)) + \beta \sum_{s' \in \mathcal{S}} P(s' \mid s, \sigma(s)) V^\sigma(s').$$

Current expected reward plus discounted expected continuation value. For a continuous state space, replace the sum by an integral with respect to $P(ds' \mid s, \sigma(s))$.

Policy Evaluation as a Linear System

From here, assume $\mathcal{S}$ and each nonempty $\mathcal{A}(s)$ are finite.

Fix σ . Recall its induced transition matrix and reward vector:

$$P_{ij}^{\sigma} = P(j \mid i, \sigma(i)), \quad r_i^{\sigma} = r(i, \sigma(i)).$$

Treat V^{σ} and r^{σ} as **column vectors**. Then

$$V^{\sigma} = r^{\sigma} + \beta P^{\sigma} V^{\sigma} \quad \Longleftrightarrow \quad (I - \beta P^{\sigma}) V^{\sigma} = r^{\sigma}.$$

- P^{σ} is row-stochastic, so $\beta < 1$ makes $I - \beta P^{\sigma}$ invertible:

$$V^{\sigma} = \sum_{t=0}^{\infty} \beta^t (P^{\sigma})^t r^{\sigma} = (I - \beta P^{\sigma})^{-1} r^{\sigma}.$$

- In code, solve the linear system rather than forming the inverse.

Example: Income Along an Employment Chain

Example: Employment and Unemployment

Take the earlier employment dynamics as given, and attach income 0 in U and 1 in E. There is no action choice in this calculation.

$$P = \begin{pmatrix} 0.8 & 0.2 \\ 0.05 & 0.95 \end{pmatrix}, \quad r = \begin{pmatrix} 0 \\ 1 \end{pmatrix}, \quad \beta = 0.9.$$

The value of the income stream solves

$$V(U) = 0 + 0.9[0.8V(U) + 0.2V(E)],$$

$$V(E) = 1 + 0.9[0.05V(U) + 0.95V(E)].$$

$$V(U) = \frac{72}{13} \approx 5.538, \quad V(E) = \frac{112}{13} \approx 8.615.$$

- ▶ Unemployment has positive value because future employment is possible.
- ▶ Employment is worth less than $1/(1 - 0.9) = 10$, because job loss is possible.

Iterative Policy Evaluation

We can also compute V^σ by repeated substitution, keeping σ fixed:

$$v^{(n+1)} = r^\sigma + \beta P^\sigma v^{(n)}, \quad v^{(0)} = 0.$$

- ▶ Use the old vector $v^{(n)}$ for every state in a sweep.
- ▶ After n sweeps, $v^{(n)}$ is the expected return from the first n rewards under σ , with zero value beyond them.

For the employment example:

$$v^{(0)} = \begin{pmatrix} 0 \\ 0 \end{pmatrix}, \quad v^{(1)} = \begin{pmatrix} 0 \\ 1 \end{pmatrix}, \quad v^{(2)} = \begin{pmatrix} 0.180 \\ 1.855 \end{pmatrix}, \quad v^{(n)} \longrightarrow \begin{pmatrix} 5.538 \dots \\ 8.615 \dots \end{pmatrix}.$$

n counts computational iterations. The policy being evaluated does not change.

From Policy Values to the Optimal Value

Let Σ be the set of feasible stationary deterministic policies. Define the **optimal value function** by

$$V^*(s) = \max_{\sigma \in \Sigma} V^\sigma(s).$$

- ▶ A policy σ^* is **optimal** if it achieves this value at every state:

$$V^{\sigma^*}(s) = V^*(s) \quad \text{for every } s \in \mathcal{S}.$$

- ▶ In a finite, time-homogeneous discounted MDP, such a stationary deterministic policy exists. Allowing randomized or history-dependent policies cannot improve the value.
- ▶ The policy may be nonunique when several actions are equally good.

The task now is to choose a policy, as well as evaluate the rewards it generates.

The Bellman Optimality Equation

The **principle of optimality**: after the next state is observed, an optimal policy uses an optimal continuation from that state.

Choose the current action a , then continue optimally. Its expected return is

$$r(s, a) + \beta \sum_{s'} P(s' \mid s, a) V^*(s').$$

Maximizing over the feasible current actions gives the **Bellman optimality equation**:

$$V^*(s) = \max_{a \in \mathcal{A}(s)} \left\{ r(s, a) + \beta \sum_{s'} P(s' \mid s, a) V^*(s') \right\}.$$

- ▶ The current action is chosen before the random next state is observed.
- ▶ Future actions can respond to that realized state. $V^*(s')$ includes all those later choices.

Example: The Cake-Eating Bellman Equation

Example: Cake Eating

Use the same whole-unit model, $k \in \{0, 1, 2\}$ and $\beta = 0.9$:

$$V^*(k) = \max_{c \in \{0, \dots, k\}} \{\sqrt{c} + 0.9V^*(k - c)\}.$$

At zero stock, $V^*(0) = 0.9V^*(0)$, so $V^*(0) = 0$. At one unit,

$$V^*(1) = \max\{0.9V^*(1), 1\} = 1.$$

At two units, check all three feasible actions:

$$V^*(2) = \max\{\underbrace{0.9V^*(2)}_{c=0}, \underbrace{1 + 0.9V^*(1)}_{c=1}, \underbrace{\sqrt{2}}_{c=2}\} = 1.9.$$

- ▶ At this solution, the three candidate values are 1.71, 1.9, 1.414...
- ▶ Thus $\sigma^*(2) = 1$, $\sigma^*(1) = 1$, and $\sigma^*(0) = 0$.

The Bellman Operator

An **operator** maps a candidate value function v to a new function.

$$(Tv)(s) = \max_{a \in \mathcal{A}(s)} \left\{ r(s, a) + \beta \sum_{s'} P(s' \mid s, a) v(s') \right\}.$$

- ▶ $v(s')$ is our current assessment of future states. Tv updates that assessment using the best current action.
- ▶ Holding a policy fixed instead gives

$$(T^\sigma v)(s) = r(s, \sigma(s)) + \beta \sum_{s'} P(s' \mid s, \sigma(s)) v(s').$$

- ▶ A **fixed point** is unchanged by its operator:

$$V^* = TV^*, \quad V^\sigma = T^\sigma V^\sigma.$$

Why the Iteration Converges

Use the **maximum norm** $\|v - w\|_\infty = \max_s |v(s) - w(s)|$. For any state and action,

$$\left| \beta \sum_{s'} P(s' | s, a) [v(s') - w(s')] \right| \leq \beta \|v - w\|_\infty.$$

Taking the maximum over actions preserves this bound, since

$$|\max_a x_a - \max_a y_a| \leq \max_a |x_a - y_a|. \text{ Hence}$$

$$\|Tv - Tw\|_\infty \leq \beta \|v - w\|_\infty.$$

- ▶ T is a **contraction**: the new distance is at most $\beta < 1$ times the old distance.
- ▶ On the finite-dimensional value space, it has a unique fixed point V^* , and

$$\|T^n v^{(0)} - V^*\|_\infty \leq \beta^n \|v^{(0)} - V^*\|_\infty.$$

- ▶ The same argument applies to T^σ and its fixed point V^σ .

Value Function Iteration

Given the finite MDP and a desired value-error tolerance $\varepsilon > 0$:

1. Choose $v^{(0)}$, for example zero, and set $n = 0$.
2. Update every state using the previous vector:

$$v^{(n+1)}(s) = (Tv^{(n)})(s) = \max_{a \in \mathcal{A}(s)} \left\{ r(s, a) + \beta \sum_{s'} P(s' | s, a) v^{(n)}(s') \right\}.$$

3. Calculate the **Bellman residual** of $v^{(n)}$:

$$d_n = \|Tv^{(n)} - v^{(n)}\|_{\infty} = \|v^{(n+1)} - v^{(n)}\|_{\infty}.$$

4. If $d_n \leq (1 - \beta)\varepsilon$, return $v^{(n)}$. Otherwise increase n and repeat.

The contraction inequality implies

$$\|v^{(n)} - v^*\|_{\infty} \leq \frac{d_n}{1 - \beta} \leq \varepsilon.$$

This bound assumes exact expectations and maximization in the specified finite model.

Example: Value Iteration for Cake Eating

Example: Cake Eating

Start from $v^{(0)}(k) = 0$, using $\beta = 0.9$.

	Stock 0	Stock 1	Stock 2
$v^{(0)}$	0	0	0
$v^{(1)} = T_V^{(0)}$	0	1	$\sqrt{2} \approx 1.414$
$v^{(2)} = T_V^{(1)}$	0	1	1.9
$v^{(3)} = T_V^{(2)}$	0	1	1.9

- ▶ The first update assigns zero continuation value, so consuming everything looks best.
- ▶ In the second update, saving one unit has value:

$$1 + 0.9v^{(1)}(1) = 1.9 > \sqrt{2}.$$

- ▶ A further update leaves the vector unchanged: the Bellman residual is zero.

A Finite Horizon: Backward Induction

If decisions end at date $H - 1$, let $h(s_H)$ be a terminal payoff at date H . The objective from $s_t = s$ is

$$\max_{\sigma_t, \dots, \sigma_{H-1}} \mathbb{E} \left[\sum_{j=t}^{H-1} \beta^{j-t} R_{j+1} + \beta^{H-t} h(s_H) \mid s_t = s \right].$$

Backward induction starts at the terminal date and works toward the present:

$$V_H(s) = h(s),$$

$$V_t(s) = \max_{a \in \mathcal{A}(s)} \left\{ r(s, a) + \beta \sum_{s'} P(s' \mid s, a) V_{t+1}(s') \right\}, \quad t = H - 1, \dots, 0.$$

- The remaining horizon changes with t ; optimal decision rules can depend on time.
- With $h = 0$, $T^H 0$ is the value with H decisions remaining. This also explains the earlier cake iterations.

Recovering a Policy from Values

Once we have V^* , choose an action that attains its Bellman maximum:

$$\sigma^*(s) \in \arg \max_{a \in \mathcal{A}(s)} \left\{ r(s, a) + \beta \sum_{s'} P(s' | s, a) V^*(s') \right\}.$$

- ▶ A **greedy policy with respect to v** uses the same expression with a candidate value v in place of V^* .
- ▶ With an approximate value, the resulting policy is a candidate. Evaluate it by solving

$$(I - \beta P^\sigma) V^\sigma = r^\sigma.$$

- ▶ The value function summarizes future rewards; the policy specifies the action to take. Computing both makes the solution usable.

References and Further Reading

1. **Richard S. Sutton and Andrew G. Barto (2018), Reinforcement Learning: An Introduction. 2nd ed. MIT Press. [Book]**
 - ▶ Chapter 3: Finite Markov Decision Processes (Sections 3.3, 3.5 and 3.6)
 - ▶ Chapter 4: Dynamic Programming (Sections 4.1 and 4.4)
2. **Thomas J. Sargent and John Stachurski, QuantEcon. [Online lectures]**
 - ▶ Discrete State Dynamic Programming
3. **Feng (2026), Foundations of Macroeconomics: ECON 282E, Fall 2026. [Online course]**
 - ▶ Session 1, Deck 1.B2: Dynamic Programming: Why It Works and How It Computes